



File Handling in Python

Reading, Writing, and Managing Files — A structured introduction for Python programmers

What Is File Handling?



File handling is the ability of a program to **create, read, update, and delete** files stored on disk. It allows data to persist beyond the runtime of a program.

→ Persistent Storage

Data survives after the program ends

→ Data Exchange

Share information between programs and systems

→ Real-World Applications

Logging, config files, reports, and more

Types of Files in Python

Text Files

Store data as human-readable characters. Each line ends with a newline character `\n`.

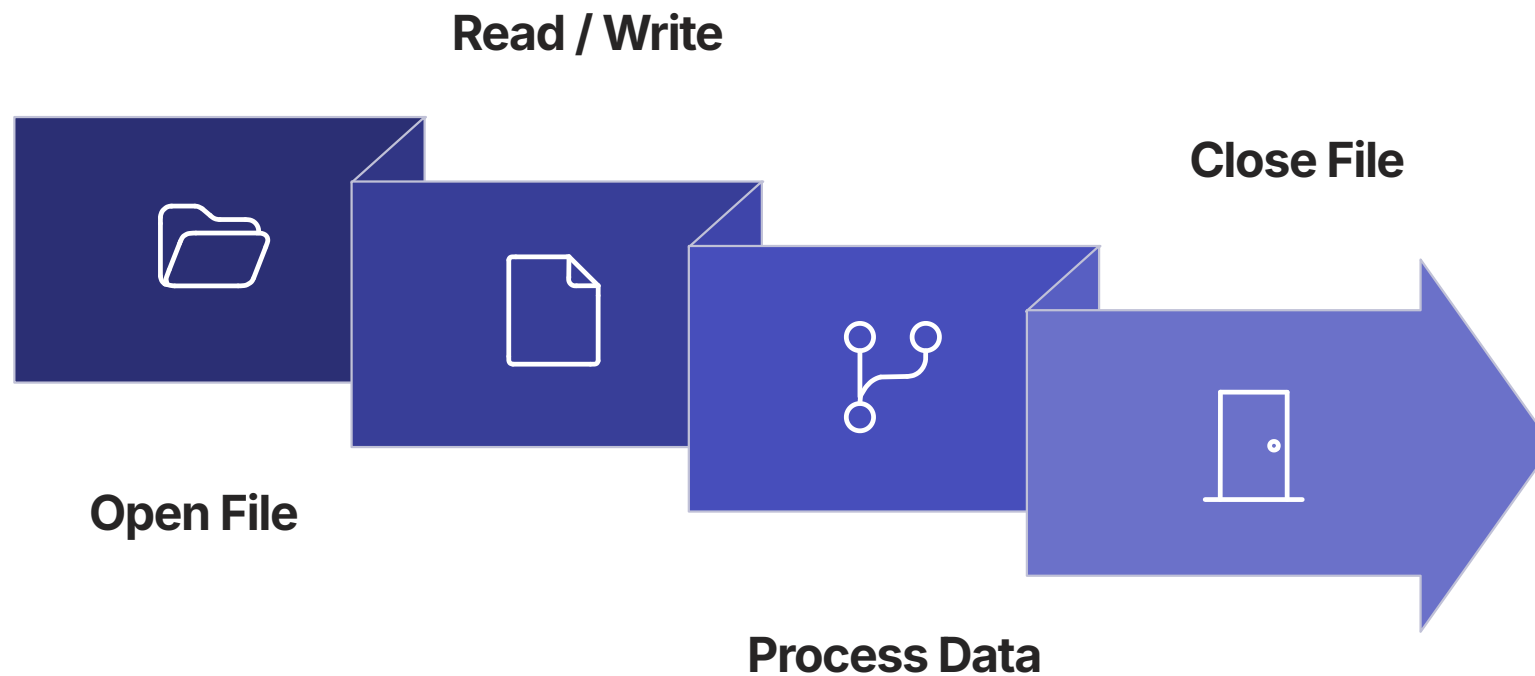
- Examples: `.txt`, `.csv`, `.log`, `.py`
- Opened in **text mode** by default
- Easy to read and edit manually

Binary Files

Store data in raw binary format — not directly readable as text. Require special handling.

- Examples: `.jpg`, `.mp3`, `.exe`, `.pdf`
- Opened with `b` flag: `"rb"`, `"wb"`
- Used for images, audio, and compiled data

Core File Operations



Every file interaction in Python follows this lifecycle. Python's `with` statement automates the close step, preventing resource leaks and making code cleaner and safer.

File Modes in Python

When opening a file with `open()`, the **mode** argument controls how the file is accessed.

Mode	Name	Behavior
"r"	Read	Opens for reading. Error if file does not exist. (Default)
"w"	Write	Opens for writing. Creates new file or overwrites existing content.
"a"	Append	Opens for writing. Adds to end of file without overwriting.
"r+"	Read+Write	Opens for both reading and writing. File must exist.
"rb" / "wb"	Binary	Read or write in binary mode for non-text files.

Reading Files

Three Ways to Read

`read()`

Reads the **entire file** as a single string. Best for small files.

`readline()`

Reads **one line at a time**. Useful for processing line by line.

`readlines()`

Returns a **list of all lines**. Each line is a separate string element.

Example

```
# Read entire file
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

```
# Read line by line
with open("data.txt", "r") as f:
    for line in f:
        print(line.strip())
```

```
# Read all lines as list
with open("data.txt", "r") as f:
    lines = f.readlines()
    print(lines[0])
```

Writing Files

Example

```
# Write a single string
with open("output.txt", "w") as f:
    f.write("Hello, Python!\n")
    f.write("File handling is useful.\n")

# Write a list of lines
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]
with open("output.txt", "w") as f:
    f.writelines(lines)

# Append without overwriting
with open("output.txt", "a") as f:
    f.write("Appended line.\n")
```

Two Write Methods

`write()`

Writes a **single string** to the file. Does not add a newline automatically — you must include `\n`.

`writelines()`

Writes a **list of strings** to the file. Like `write()`, newlines must be included explicitly.

- ❏ Always use `"w"` to overwrite or `"a"` to append. Using `"w"` on an existing file permanently erases its content.

Complete Example Program

```
# Writing to a file
```

```
with open("example.txt", "w") as file:
```

```
    file.write("Hello, this is file handling in Python.\n")
```

```
    file.write("Data persists after the program ends.\n")
```

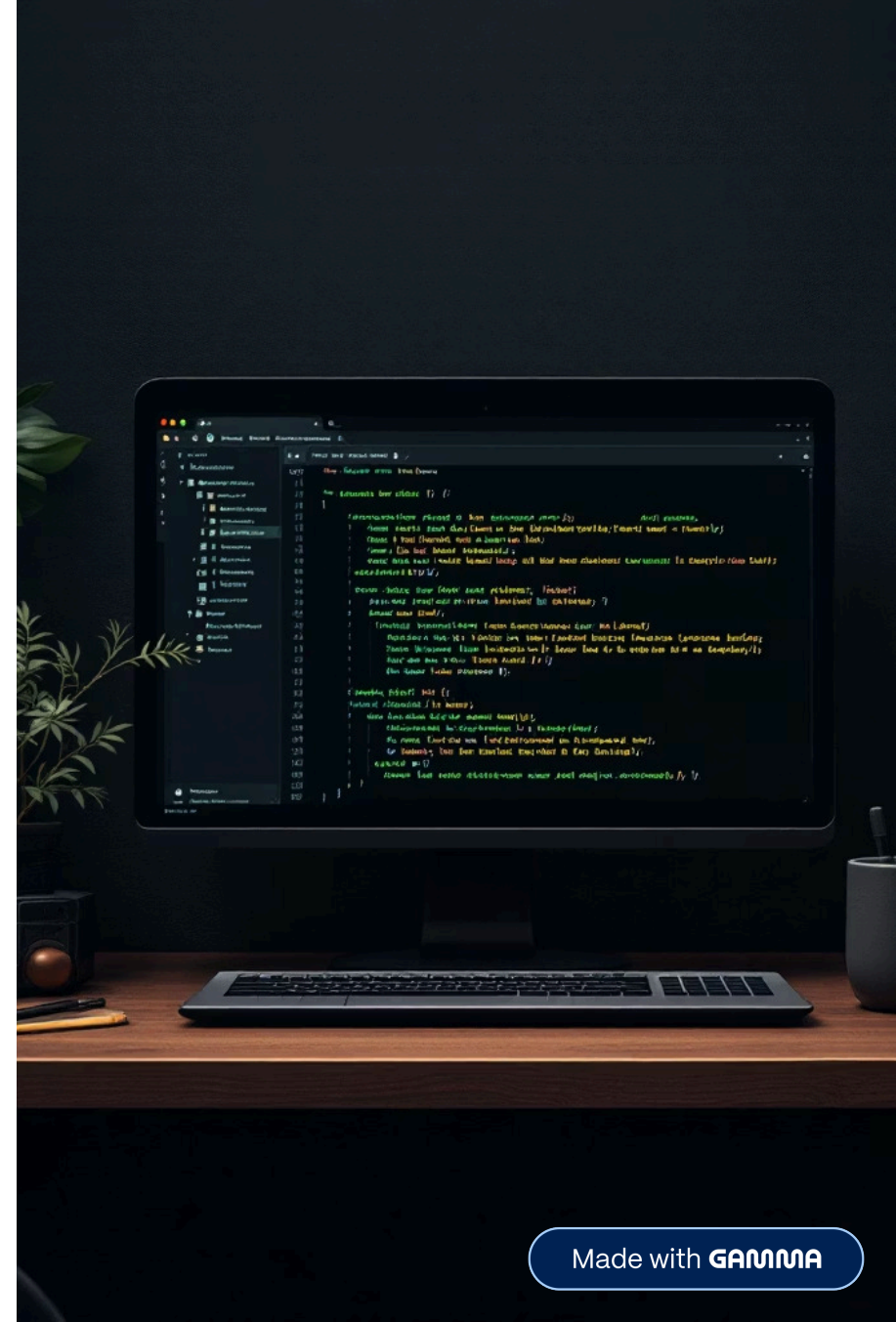
```
# Reading from the file
```

```
with open("example.txt", "r") as file:
```

```
    content = file.read()
```

```
    print(content)
```

The `with` statement automatically closes the file when the block exits — even if an error occurs. This is the **recommended pattern** for all file operations in Python.



Advantages & Use Cases



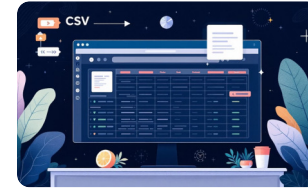
Logging

Record application events, errors, and activity to a log file for debugging and monitoring.



Configuration Files

Read settings from `.ini`, `.json`, or `.cfg` files to customize application behavior without changing code.



Data Import / Export

Process CSV or text-based datasets and export results — a foundation for data science workflows.

Key Takeaways

01

File handling enables persistence

Data written to a file outlives the program that created it.

02

Use the right mode

"r" to read, "w" to overwrite, "a" to append, "rb"/"wb" for binary.

03

Choose your read method wisely

read() for full content, readline() for one line, readlines() for a list.

04

Always use with open()

The context manager ensures files are properly closed — even when exceptions occur.

05

Real-world applications are everywhere

Logging, config management, data pipelines — file I/O is a foundational skill for every Python developer.